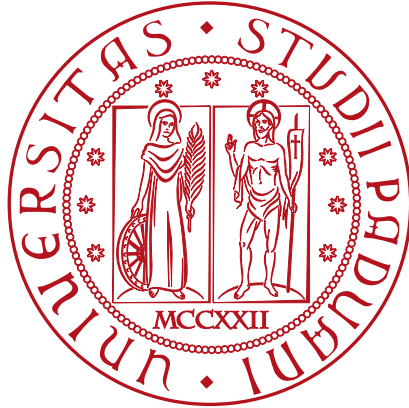


Università degli Studi di Padova

DIPARTIMENTO DI MATEMATICA "TULLIO LEVI-CIVITA"

CORSO DI LAUREA IN INFORMATICA



**Migrazione al protocollo MQTT in un
sistema di monitoraggio e tracciamento
di linee produzione basato su dispositivi
RFID**

Tesi di Laurea Triennale

Relatore

Prof. Marco Zanella

Laureando

Luca Ribon

Matricola 2075516

ANNO ACCADEMICO 2025-2026

Cit...

— Autore

Dedicato a ...

Sommario

Il presente documento descrive il lavoro svolto durante il periodo di stage, della durata di circa 320 ore, dal laureando Luca Ribon presso l'azienda KanbanBOX S.r.l.

L'obiettivo finale dello stage è stato quello di sostituire il protocollo HTTP_G, utilizzato dalla piattaforma KanbanBOX per comunicare con i lettori RFID_G, con MQTT_G, un protocollo più adatto ad essere utilizzato con dispositivi *IoT*.

In particolare è stata progettata un'architettura capace di integrare i seguenti componenti:

- **lettore RFID**, dispositivi hardware che leggono i tag RFID applicati sui prodotti;
- **AWS IoT Core**, un servizio *cloud* che integra un broker_G capace di gestire la comunicazione tra molteplici dispositivi tramite MQTT;
- **AWS SQS**, un servizio *cloud* che fornisce delle code di messaggi per la comunicazione asincrona tra servizi, particolarmente adatto a gestire la lettura dei tag tramite meccanismo *publish/subscribe* di MQTT;
- **KanbanBOX**, la piattaforma *web* che necessita di ricevere i dati letti dai lettori RFID per monitorare e supportare l'intero processo produttivo e logistico di un'azienda.

A tal fine, alla fase di progettazione è seguita la codifica delle funzionalità necessarie in KanbanBOX per l'integrazione con i servizi AWS IoT Core e SQS, che saranno di supporto per l'implementazione della comunicazione con i lettori tramite protocollo MQTT.

Ringraziamenti

Innanzitutto, vorrei esprimere la mia gratitudine al Prof. Marco Zanella relatore della mia tesi, per l'aiuto e il sostegno fornitomi durante tutto il processo di stesura.

Desidero ringraziare con affetto la mia famiglia per il sostegno, il grande aiuto e per essermi stati vicini in ogni momento durante gli anni di studio.

Infine tengo a ringraziare tutti i miei amici per aver contribuito a rendere memorabile ogni momento passato assieme, che fosse di svago o produttivo.

Padova, Aprile 2026

Luca Ribon

Indice

1. Introduzione	1.
1.1. Organizzazione del testo (WIP)	1.
1.2. L'azienda	1.
1.3. Il progetto	1.
2. Descrizione dello stage	4.
2.1. Contesto aziendale	4.
2.2. Metodo di lavoro	4.
2.2.1. Pianificazione e revisione delle attività	4.
2.2.1.1. Git e GitHub	4.
2.2.1.1.1. Integrazione continua	5.
2.2.1.1.2. Distribuzione continua	6.
2.2.1.2. Sentry	6.
2.2.2. Altri strumenti a supporto dello sviluppo	6.
2.2.2.1. PhpStorm	6.
2.2.2.2. Docker	6.
2.2.2.3. Microsoft Teams	7.
2.3. Analisi dei rischi	7.
2.3.1. Rischi organizzativi	7.
2.3.1.1. Gestione del tempo non ottimale	7.
2.3.2. Rischi tecnici	7.
2.3.2.1. Inesperienza con tecnologie usate in azienda	7.
2.3.2.2. Influenze esterne	7.
2.3.3. Rischi di analisi e progettazione	7.
2.3.3.1. Variazione dei requisiti	8.
2.3.3.2. Scelte progettuali non adeguate	8.
3. Analisi dei requisiti	10.
4. Introduzione teorica	12.
4.1. Tecnologie analizzate	12.
4.2. Aspetti teorici rilevanti	12.
4.3. Strumenti scelti	12.
5. Descrizione del lavoro svolto	14.
6. Conclusioni	16.
6.1. Consuntivo finale	16.
6.2. Raggiungimento degli obiettivi	16.
6.3. Conoscenze acquisite	16.

6.4. Probabili margini di miglioramento	16.
6.5. Considerazioni personali	16.
Bibliografia	18.
Glossario	20
A	20
API	20
B	20
Broker	20
C	20
D	20
E	20
F	20
G	20
H	20
HTTP	20
I	20
IDE	20
J	20
K	20
Kanban	20
L	21
Lean	21
M	21
MQTT	21
N	21
O	21
P	21
Q	21
R	21
RFID	21
S	21
T	21
U	21
V	21
W	21
Word	21
X	21
Y	21
Z	21

Elenco delle Figure

Elenco delle Tabelle

Capitolo 1.

Introduzione

1.1. Organizzazione del testo (WIP)

Il documento è strutturato in sei capitoli principali:

1. Introduzione: describe.
2. Descrizione dello stage: describe.
3. Analisi dei requisiti: describe.
4. Introduzione teorica: describe.
5. Descrizione del lavoro svolto: describe.
6. Conclusioni: describe.

Riguardo la stesura del testo, sono state adottate le seguenti convenzioni tipografiche:

- gli acronimi, le abbreviazioni e i termini ambigui o di uso non comune vengono definiti nel glossario, situato alla fine del presente documento; per la prima occorrenza dei termini riportati nel glossario viene utilizzato il seguente stile: termine_G ;
- i termini in lingua straniera o facenti parti del gergo tecnico sono evidenziati con il carattere *corsivo*.

1.2. L'azienda

KanbanBOX S.r.l. è un'azienda con sede a Vicenza che si occupa dello sviluppo, della commercializzazione e della consulenza relativi alla piattaforma web KanbanBOX.

KanbanBOX è uno strumento progettato per supportare le aziende nell'implementazione di metodologie Lean_G nella gestione della produzione e della logistica delle aziende. Più nello specifico, la piattaforma consente di tracciare e gestire lo stato dei materiali grezzi, semilavorati e finiti, durante le fasi di approvvigionamento e produzione. Ciò è reso possibile attraverso l'utilizzo di cartellini Kanban_G elettronici, che permettono di identificare e tracciare i prodotti man mano che vengono consumati o resi disponibili dai processi aziendali.

1.3. Il progetto

KanbanBOX prevede due metodi di lettura dei cartellini Kanban:

- scansione di **codici a barre**: per ogni cartellino vengono stampata un'etichetta su cui sono presenti dei codici a barre che possono essere scansionati tramite appositi lettori ottici o tramite la fotocamera di comuni dispositivi mobili;
- lettura di chip **RFID**: per ogni cartellino viene stampata un'etichetta RFID, ovvero un'etichetta dotata di un *chip* che può essere letto da apposite antenne RFID.

In entrambi i casi le operazioni svolte sui cartellini vengono comunicate a KanbanBOX tramite delle API_G RESTful che sfruttano il protocollo HTTP per la trasmissione dei dati. In particolare, nel caso della lettura tramite RFID, il lettore RFID mette a disposizione delle API HTTP, eseguite in locale sul lettore stesso, che consentono la comunicazione con esso.

I modelli di lettori RFID utilizzati da KanbanBOX supportano anche la comunicazione tramite protocollo MQTT, un protocollo di comunicazione del livello applicativo, basato su un meccanismo *publish/subscribe*; in aggiunta i lettori implementano nativamente un metodo di connessione sicuro e pratico al servizio AWS IoT Core.

MQTT è particolarmente adatto per la comunicazione con dispositivi *IoT*, in quanto progettato per essere meno oneroso in termini di consumo di banda e risorse computazionali rispetto a HTTP; inoltre prevede dei meccanismi di *Quality of Service (QoS)* e di memorizzazione dei messaggi che lo rendono più affidabile in scenari in cui la connettività di rete non è necessariamente stabile.

Per questi motivi il progetto di stage si è focalizzato sulla sostituzione del protocollo HTTP con MQTT per la trasmissione dei tag letti tra lettori RFID e KanbanBOX con l'obiettivo di migliorare l'affidabilità e l'efficienza della comunicazione; inoltre è stata prevista l'implementazione di funzionalità di configurazione dei lettori direttamente all'interno di KanbanBOX, operazioni che in precedenza dovevano essere eseguite manualmente tramite l'interfaccia web fornita dal produttore del lettore.

Ho scelto questo progetto perché prometteva di approfondire e di maturare esperienza con temi e tecnologie che mi incuriosiscono, come l'*Internet of Things*, i protocolli di comunicazione e le infrastrutture *cloud*. Un altro fattore interessante è stato il contatto diretto e concreto con il prodotto su cui ho lavorato che ho potuto vedere all'opera in un contesto reale.

Capitolo 2.

Descrizione dello stage

2.1. Contesto aziendale

Lo stage si è svolto in presenza presso la sede di Vicenza di KanbanBOX S.r.l. Sono stato inserito nel team sviluppatori software con il quale ho, fin da subito, partecipato a tutte le attività di routine previste per tutto il gruppo.

Per tutta la durata dello stage ho avuto a disposizione il supporto del mio tutor aziendale e del mio *buddy*, i quali mi hanno guidato nell’inserimento all’interno dell’azienda, del team e del prodotto KanbanBOX su cui ho lavorato.

Inoltre, ho avuto modo di collaborare anche con altri membri del team per discutere con chi aveva maggiore esperienza in merito agli argomenti o alle tecnologie con cui ero meno familiare.

Oltre al supporto *peer-to-peer* ricevuto ho potuto seguire diversi corsi di formazione interna organizzati dall’azienda per approfondire i concetti, sia teorici che pratici, legati alla metodologia Lean, ai cartellini Kanban e all’utilizzo di KanbanBOX.

2.2. Metodo di lavoro

2.2.1. Pianificazione e revisione delle attività

Per il team di sviluppo sono previste diverse occasioni ricorrenti per la pianificazione e la revisione delle attività svolte:

- **Daily stand-up meeting**: riunione giornaliera di circa 15 minuti in cui ogni membro del team condivide lo stato di avanzamento del proprio lavoro, le difficoltà incontrate e gli obiettivi per la giornata corrente;
- **Dev+ weekly**: riunione settimanale del team di sviluppo a cui partecipa anche un consulente esterno, con esperienza rilevante nel mondo PHP *open source*, con cui si affrontano i problemi più ostici e si analizzano approfonditamente potenziali soluzioni;
- **Analisi issue Sentry**: riunione settimanale tra tutti i membri del team in cui si analizza il report delle *issue* tracciate da Sentry, uno strumento di monitoraggio degli errori in produzione; l’obiettivo dell’incontro è quello di identificare i bug più critici e frequenti e di assegnarli ai membri del team per la risoluzione;
- **OKR (Objective and Key Results)**: riunione svolta su base trimestrale in cui, partendo da degli obiettivi aziendali più generici, si definiscono gli obiettivi specifici per il team di sviluppo e i risultati chiave che ne misurano il raggiungimento. Così facendo si assicura che ogni team si impegni a supportare gli obiettivi aziendali e non solo le proprie esigenze interne o del cliente.

2.2.1.1. Git e GitHub

Per il **versionamento** di tutta la *codebase* e la documentazione di KanbanBOX viene utilizzato **Git**, un sistema di controllo di versione distribuito.

I repository Git sono ospitati su **GitHub**, una piattaforma utilizzata per la gestione del codice sorgente e la collaborazione tra sviluppatori, che fornisce funzionalità aggiuntive, oltre al semplice versionamento, usate dal team di KanbanBox come la gestione delle *issue*, le *pull request* con relative verifiche del codice e l'integrazione continua.

Nel repository di KanbanBOX, ovvero quello contenente tutta la *codebase* del prodotto, viene usato il flusso di lavoro **Git Flow**, che prevede l'utilizzo di due rami principali, **master** e **development**.

In **master** viene mantenuta sempre la versione stabile e in produzione, mentre in **development** viene costruito lo *snapshot* della prossima *release*. In aggiunta a questi due rami principali, ne esistono altri con scopi specifici che seguono la seguente nomenclatura:

- **feature/nome-feature**: rami usati per lo sviluppo di nuove funzionalità;
- **fix/nome-fix**: rami usati per la risoluzione di bug;
- **hotfix/nome-hotfix**: rami usati per la correzione di bug critici in produzione;
- **chore/nome-chore**: rami usati per attività di manutenzione o che in generale non apportano modifiche perceibili dall'utente finale (aggiornamenti di librerie, configurazioni, implementazione di funzionalità esistenti che non prevedono risoluzione di bug).

Quando lo scopo di un ramo viene assolto interamente, viene aperta una **pull request** su GitHub per richiedere la revisione e l'integrazione delle modifiche nel ramo **development** (o **master** nel caso di hotfix).

Ad ogni pull request si applicano delle **politiche di revisione** in base all'impatto che l'*issue* risolta ha sul prodotto:

- *low*: nessuna revisione richiesta;
- *average*: revisione da parte di almeno un altro sviluppatore;
- *high*: revisione da parte di almeno due altri sviluppatori.

Quando si individuano problemi o nuove esigenze internamente al team vengono create delle *issue* su GitHub dai membri interessati seguendo un modello predefinito che ne facilita la categorizzazione e la prioritizzazione.

Oltre agli sviluppatori, anche il team dei consulenti applicativi utilizza i repository su GitHub per svolgere le attività di supporto ai clienti; infatti, ogni qualvolta viene aperta una richiesta di assistenza o di una nuova funzionalità da parte di un cliente, questa viene trasformata in un'*issue* su GitHub in cui vengono documentati i dettagli della richiesta ricevuta.

Infine le *issue* possono anche essere il risultato delle riunioni OKR.

L'assegnazione delle *issue* ai singoli membri avviene su base volontaria; ci sono però dei casi in cui l'assegnazione viene fatta in modo diretto, solitamente quando si hanno delle scadenze stringenti, così da essere sicuri di assegnare le *issue* ai membri con maggiore *ownership* tecnica sull'argomento trattato.

Inoltre, capita di scegliere volontariamente di mantenere certe *issue* a granularità meno fine, così che stia agli sviluppatori con più esperienza la decisione sul come suddividerle o se assegnarle ad un sotto-team dedicato.

2.2.1.1.1. Integrazione continua

Per ogni pull request aperta su GitHub vengono eseguite automaticamente le seguenti GitHub **Actions**:

- **PHPUnit**: esecuzione di test di unità, di integrazione ed *end-to-end*;
- **Playwright**: esecuzione di test *end-to-end* che simulano le operazioni utente tramite *browser* automatizzati;

- **PSALM**: analisi statica del codice PHP per individuare potenziali *bug* e problemi di sicurezza.

Tutti i test vengono eseguiti sia simulando il nuovo prodotto nella sua interezza, sia nelle seguenti casistiche meno comuni:

- esecuzione di nuove implementazioni o integrazioni nel codice in ambienti con le configurazioni della release stabile corrente; questo permette di individuare potenziali **problemi di retrocompatibilità**;
- esecuzione del vecchio codice con in ambienti con le nuove configurazioni previste per la prossima release; questo permette di misurare la capacità di fare **rollback** in caso di problemi con la nuova release.

Anche per i test vengono usati dei *container* in modo da isolare l'ambiente di esecuzione e garantire che i test siano più semplici da automatizzare e sempre riproducibili in modo consistente.

2.2.1.1.2. Distribuzione continua

Giornalmente viene distribuita una **release** che include tutte le pull request approvate e integrate nel ramo **development**.

Rilevante è l'impegno che KanbanBOX sta impiegando per automatizzare il più possibile il processo di distribuzione, sfruttando, anche in questo caso, i Docker *container*.

2.2.1.2. Sentry

Sentry è una piattaforma di **monitoraggio degli errori** che consente di tracciare, analizzare e risolvere i problemi nelle applicazioni in tempo reale.

All'interno di KanbanBOX viene utilizzata sia per ricevere notifiche automatiche sugli errori che si verificano in produzione, sia per gestire in modo più ordinato e collaborativo gli errori che si verificano aggiungendo dettagli utili ai *bug* su cui si sta lavorando e categorizzandoli.

Giornalmente viene scelto un membro del team di sviluppo che si dedicherà maggiormente a monitorare e analizzare le issue segnalate da Sentry in quella giornata. In più, settimanalmente viene organizzata una riunione dove il **responsabile Sentry** di quella giornata, assieme al resto del team, analizza e discute le issue più frequenti dell'ultimo periodo con l'obiettivo di categorizzarle, documentarle nel modo più dettagliato possibile e assegnarle ai membri del team per la risoluzione.

2.2.2. Altri strumenti a supporto dello sviluppo

2.2.2.1. PhpStorm

Per lo sviluppo di KanbanBOX viene utilizzato PhpStorm, un **ambiente di sviluppo integrato (IDE_G)** specifico per il linguaggio PHP che offre funzionalità avanzate come l'analisi statica del codice, *refactoring* automatizzato, il *debugging* e l'integrazione con sistemi di controllo di versione come Git.

2.2.2.2. Docker

Tutti i servizi che compongono KanbanBOX, o che ne supportano lo sviluppo e il *testing*, sono eseguiti in dei *container*.

Docker è la piattaforma utilizzata per creare, distribuire e gestire questi container in

modo efficiente; nel caso di KanbanBOX vengono utilizzati dei Dockerfile per istanziare e orchestrare i container, e dei file `docker-compose.yml` per definire la loro configurazione.

2.2.2.3. Microsoft Teams

Per la **comunicazione** interna al team e con il resto dell'azienda viene utilizzato Microsoft Teams, una piattaforma di collaborazione che integra chat, videoconferenze, condivisione di file e integrazione con altre applicazioni Microsoft 365.

All'interno dell'azienda sono presenti sia canali generali per la comunicazione tra tutti i dipendenti, sia canali specifici per ogni team di lavoro. Inoltre sono presenti canali dedicati ad attività specifiche come le riunioni Dev+ weekly e le analisi delle issue di Sentry (vedi Sezione 2.1).

2.3. Analisi dei rischi

I rischi individuati e analizzati nella fase iniziale del progetto si possono suddividere in tre categorie principali:

- **rischi organizzativi;**
- **rischi tecnici;**
- **rischi di analisi e progettazione.**

2.3.1. Rischi organizzativi

2.3.1.1. Gestione del tempo non ottimale

Descrizione: durante lo svolgimento dello stage potrebbe non essere sempre possibile gestire il tempo in modo ottimale a causa di attività aziendali ausiliarie o imprevisti.

Soluzione: pianificazione intelligente degli appuntamenti di importanza secondaria in modo che non vadano a interferire con le attività principali dello stage.

2.3.2. Rischi tecnici

2.3.2.1. Inesperienza con tecnologie usate in azienda

Descrizione: durante lo svolgimento dello stage potrebbero presentarsi dei rallentamenti dovuti alla mia inesperienza con alcune tecnologie o metodologie usate in azienda.

Soluzione: dedicare la fase iniziale alla comprensione approfondita delle tecnologie ed interfacciarsi con i membri del team per chiedere supporto in caso di difficoltà bloccanti.

2.3.2.2. Influenze esterne

Descrizione: durante lo svolgimento dello stage potrebbero sorgere problemi tecnici dovuti a fattori esterni all'azienda, come ad esempio bug o disservizi di dipendenze esterne del prodotto.

Soluzione: analizzare il problema per identificare il prima possibile la reale causa, successivamente cercare di mitigarlo, se necessario coinvolgendo altri membri del team con maggiore esperienza; nel caso in cui il problema risultasse non risolvibile internamente ripianificare le attività in modo da minimizzare l'impatto sullo svolgimento dello stage.

2.3.3. Rischi di analisi e progettazione

I rischi appartenenti a questa categoria derivano principalmente dal fatto che le tecnologie usate non sono ancora approfonditamente esplorate dal team di sviluppo,

quindi è necessaria una fase di studio e sperimentazione per comprenderne al meglio le potenzialità, i limiti e le opzioni di integrazione con il prodotto.

2.3.3.1. Variazione dei requisiti

Descrizione: durante la fase di studio potrebbero emergere nuove informazioni che potrebbero essere incongruenti con le aspettative o rendere obsoleti i requisiti inizialmente definiti.

Soluzione: mantenere una comunicazione costante con il tutor aziendale per discutere eventuali cambiamenti nei requisiti.

2.3.3.2. Scelte progettuali non adeguate

Descrizione: durante la fase di integrazione e sviluppo potrebbero emergere nuove esigenze, limitazioni od opportunità date dalle tecnologie scelte.

Soluzione: intervallare le fasi di integrazione o sviluppo con momenti di studio e sperimentazione mirati in modo da accertarsi che le scelte fatte siano adeguate, sfruttando anche le nuove informazioni derivanti dalle fasi di integrazione o sviluppo di altre tecnologie, svolte precedentemente.

Capitolo 3.

Analisi dei requisiti

Capitolo 4.

Introduzione teorica

4.1. Tecnologie analizzate

4.2. Aspetti teorici rilevanti

4.3. Strumenti scelti

Breve introduzione al capitolo

Capitolo 5.

Descrizione del lavoro svolto

Capitolo 6.

Conclusioni

6.1. Consuntivo finale

6.2. Raggiungimento degli obiettivi

6.3. Conoscenze acquisite

6.4. Probabili margini di miglioramento

6.5. Considerazioni personali

Bibliografia

Glossario

A

API

Application Programming Interface, è un insieme di regole e protocolli che consentono a diverse applicazioni software di comunicare tra loro.

Le API definiscono i metodi e i formati per trasmettere richieste e risposte tra componenti software o hardware, facilitando la comunicazione tra sistemi diversi.

B

Broker

Un broker è un componente software che funge da intermediario nella comunicazione tra dispositivi in un'architettura publish/subscribe, come quella utilizzata dal protocollo MQTT.

C

D

E

F

G

H

HTTP

Hypertext Transport Protocol, è un protocollo di comunicazione basato su architettura client-server, utilizzato principalmente per la trasmissione di dati sul web.

È il protocollo che è stato utilizzato fin dall'inizio in KanbanBOX per la comunicazione tra i lettori RFID e la piattaforma stessa.

I

IDE

J

K

Kanban

Kanban è un sistema di gestione della produzione e del flusso di lavoro che utilizza schede visive (chiamate «kanban») per rappresentare le attività, i materiali o i prodotti in un processo produttivo.

L

Lean

Lean è una metodologia di gestione della produzione e dei processi aziendali che mira a massimizzare il valore per il cliente riducendo gli sprechi e migliorando l'efficienza. Originariamente sviluppata nel settore manifatturiero, la filosofia Lean si basa su principi come il miglioramento continuo, il rispetto per le persone e l'ottimizzazione del flusso di lavoro.

M

MQTT

Message Queuing Telemetry Transport, è un protocollo di messaggistica basata su un architettura publish/subscribe, progettato per essere leggero ed efficiente, particolarmente adatto per dispositivi con risorse limitate e reti con larghezza di banda ridotta.

N

O

P

Q

R

RFID

Radio-Frequency Identification, è una tecnologia di identificazione che utilizza onde radio per trasmettere dati tra un lettore e un dispositivo chiamato «tag» o «etichetta» RFID.

S

T

U

V

W

Word

X

Y

Z